

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

Konkurentno računarstvo – paralelno programiranje

**UPOREDNA ANALIZA SEKVENCIJALNOG I
PARALELNOG PROGRAMIRANJA U PYTHON
OKRUŽENJU
(Seminarski rad)**

**Predmetni nastavnik
Prof.dr Negovan Stamenković**

**Student: Pavlović Ivan
Br. Indeksa : 92-20/RITP-S**

Banja Luka, septembar 2026

SADRŽAJ

UVOD	1
1. PYTHON KAO PROGRAMSKO OKRUŽENJE.....	2
1.1. Istorijski razvoj programskog jezika Python	2
1.2. Primjena Pythona u industriji i nauci.....	2
1.3. Ograničenja sekvencijalnog programiranja	3
1.4. Višejezgreni procesori i potreba za paralelnim programiranjem	3
2. PARALELIZAM U PROGRAMIRANJU	5
2.1. Osnovni pojmovi paralelizma	5
2.1.1. Procesi.....	5
2.1.2. Niti	6
2.1.3. Konkurentnost.....	6
2.1.4. Paralelizam.....	6
2.2. Biblioteke i alati za paralelno programiranje u Pythonu	6
2.2.1. Threading	6
2.2.2. Multiprocessing.....	7
2.2.3. Concurrent.futures	7
2.3. Prednosti paralelnog programiranja	7
2.4. Kada koristiti paralelizam	8
2.5. Ograničenja i izazovi paralelnog programiranja.....	8
3. PRAKTIČNI PRIMJERI I POREĐENJE.....	9
3.1. Primjer 1: Sabiranje elemenata velike liste.....	9
3.2. Primjer 2: Izračunavanje kvadrata brojeva	10
3.3. Primjer 3: Provjera prostih brojeva.....	11
3.4. Primjer 4: Pretraga elementa u velikoj listi.....	14
3.5. Primjer 5: Sortiranje više nezavisnih lista	16
3.6. Primjer 6: Brojanje riječi u tekstovima	18
3.7. Primjer 7: Rad sa više tekstualnih fajlova.....	20
3.8. Primjer 8: Simulacija mrežnih zahtjeva	22
3.9. Primjer 9: Obrada slike predstavljene matricom piksela	23
3.10. Primjer 10: Monte Carlo simulacija za približnu vrijednost broja pi	25
3.11. Primjer 11: Množenje matrica	27
3.12. Primjer 12: Izračunavanje faktoriijela.....	29
3.13. Primjer 13: Filtriranje velikog skupa podataka.....	31

3.14. Primjer 14: Generisanje hash vrijednosti	33
3.15. Primjer 15: Simulacija više nezavisnih zadataka.....	35
4. PREGLED REZULTATA	37
ZAKLJUČAK.....	38
POPIS SLIKA.....	39
CITATNI IZVORI.....	40

UVOD

Razvoj savremenih računarskih sistema usko je povezan sa potrebom za sve bržom obradom podataka. U ranijim fazama razvoja računarstva, programi su se uglavnom izvršavali sekvencijalno, što znači da je procesor izvršavao jednu instrukciju za drugom. Takav način programiranja je jednostavan za razumijevanje, lakši za testiranje i pogodan za veliki broj svakodnevnih zadataka. Međutim, sa rastom količine podataka i složenosti problema, sekvencijalno programiranje sve češće postaje ograničavajući faktor.

Savremeni računari najčešće imaju više procesorskih jezgara. To znači da hardver pruža mogućnost da se više zadataka izvršava istovremeno. Da bi softver mogao iskoristiti takve mogućnosti, potrebno je primijeniti tehnike paralelnog programiranja. Paralelno programiranje omogućava da se veći problem podijeli na više manjih zadataka koji se mogu izvršavati istovremeno. Na taj način se može smanjiti vrijeme izvršavanja programa i bolje iskoristiti raspoloživi hardverski resursi.

U ovom radu obrađuje se uporedna analiza sekvencijalnog i paralelnog programiranja u Python okruženju. Python je izabran jer je jednostavan za čitanje, široko primjenjiv i posjeduje više biblioteka koje omogućavaju konkurentno i paralelno izvršavanje programa. Zvanična dokumentacija opisuje Python kao programski jezik koji je jednostavan za učenje, ali istovremeno dovoljno moćan za ozbiljan razvoj softvera

Cilj rada je da se kroz teorijski dio objasne osnovni pojmovi paralelizma, a zatim kroz praktične primjere prikaže razlika između sekvencijalnog i paralelnog pristupa. Posebna pažnja posvećena je mjerenju vremena izvršavanja programa i analizi situacija u kojima paralelizam donosi ubrzanje, ali i situacija u kojima paralelna verzija može biti jednaka ili čak sporija od sekvencijalne.

1. PYTHON KAO PROGRAMSKO OKRUŽENJE

1.1. Istorijski razvoj programskog jezika Python

„Python je programski jezik visokog nivoa koji je kreirao Guido van Rossum krajem osamdesetih i početkom devedesetih godina dvadesetog vijeka. Prema zvaničnoj Python dokumentaciji, Python je nastao početkom devedesetih godina u institutu CWI u Holandiji kao nasljednik jezika ABC. Prva javna verzija Pythona objavljena je 1991. godine.

Python spada u interpretirane jezike opšte namjene koji, zahvaljujući jednostavnoj sintaksi i čitljivosti koda, omogućava studentima brzo savladavanje osnovnih programerskih koncepata bez potrebe za dubljim poznavanjem niskonivojskih detalja rada računara. Autor ističe da je upravo ta pristupačnost jedan od razloga zašto se Python sve češće uvodi kao prvi jezik u nastavi programiranja na tehničkim fakultetima.

Već u ranim verzijama Python je imao osobine koje su ga razlikovale od mnogih drugih jezika, a to su jednostavna sintaksa, čitljiv kod, podrška za module i rad sa osnovnim strukturama podataka. Tokom vremena Python se razvijao kroz verzije Python 1, Python 2 i Python 3. Danas je Python 3 standardna verzija jezika i koristi se u obrazovanju, industriji, nauci i istraživanju.

Jedna od najvažnijih osobina Pythona jeste njegova čitljivost. Kod napisan u Pythonu često izgleda jednostavnije nego kod u jezicima kao što su C++ ili Java. U pomenutoj skripti se posebno naglašava da upotreba uvlačenja (indentation) umjesto viličastih zagrada za obilježavanje blokova koda doprinosi tome da Python program liči na strukturiran tekst, što olakšava i učenje i kasnije održavanje koda. Zbog toga je Python pogodan za početnike, ali i za profesionalne programere koji žele brzo razvijati prototipove, skripte, alate i kompleksnije aplikacije“ [1].

Python takođe ima veliku standardnu biblioteku i veliki broj dodatnih paketa, što značajno proširuje njegovu upotrebu od obrade signala i telekomunikacionih simulacija, oblasti kojom se bavi i sama skripta ETF-a, do veb razvoja, analize podataka i mašinskog učenja.

1.2. Primjena Pythona u industriji i nauci

„Python se danas koristi u velikom broju oblasti. U industriji se često koristi za automatizaciju, web razvoj, analizu podataka, testiranje softvera i razvoj skripti koje olakšavaju svakodnevni rad. Njegova popularnost dolazi iz činjenice da omogućava brz razvoj i jednostavno povezivanje sa drugim tehnologijama. U nauci Python ima posebno značajnu ulogu. Biblioteke kao što su NumPy, pandas, SciPy i matplotlib omogućavaju rad sa velikim skupovima podataka, matematičke proračune, statističku analizu i vizuelizaciju rezultata. U oblasti vještačke inteligencije i mašinskog učenja Python se koristi zajedno sa bibliotekama kao što su TensorFlow, PyTorch i scikit-learn. Python je pogodan i za eksperimentalno programiranje. Kada istraživač želi brzo provjeriti neku ideju, Python omogućava da se algoritam napiše u relativno kratkom vremenu“ [2].

Upravo ova kombinacija jednostavnosti i velikog broja biblioteka čini Python jednim od najčešće korišćenih jezika u savremenom softverskom razvoju, kako u industrijskom, tako i u akademskom i istraživačkom okruženju.

1.3. Ograničenja sekvencijalnog programiranja

Sekvencijalno programiranje podrazumijeva da se instrukcije izvršavaju redom, jedna za drugom. Takav pristup je prirodan i jednostavan za razumijevanje. Na primjer, ako program treba da obradi hiljadu brojeva, sekvencijalna verzija će uzeti prvi broj, obraditi ga, zatim drugi, zatim treći i tako dalje.

„Problem se javlja kada se obrađuju veliki skupovi podataka ili kada jedan zadatak može trajati veoma dugo. Ako računar ima više procesorskih jezgara, sekvencijalni program obično koristi samo jedno jezgro za glavni tok izvršavanja. Ostala jezgra tada ostaju djelimično neiskorišćena. To znači da hardver ima mogućnost za brži rad, ali program nije napisan tako da tu mogućnost iskoristi.

Ovo ograničenje je posebno izraženo u standardnoj implementaciji Pythona (CPython). CPython koristi mehanizam poznat kao globalni interpreterski katanac (Global Interpreter Lock, GIL) jedinstvenu bravu koju nit mora posjedovati da bi mogla pristupati Python objektima“ [3]. Bez ove brave, čak i naizgled jednostavne operacije mogle bi izazvati probleme u višenitnom programu, jer bi dvije niti istovremeno mijenjale isti objekat na nepredvidiv način. Posljedica je da, iako je moguće pokrenuti više niti unutar jednog Python procesa, samo jedna od njih u datom trenutku zaista izvršava Python bajtkod što znači da klasično višenitno programiranje u CPython-u ne donosi pravi paralelizam za zadatke koji intenzivno koriste procesor.

„GIL predstavlja jednu od glavnih prepreka efikasnom korišćenju višejezgarnih procesora iz Pythona, a ovaj problem posebno pogađa naučno-računarske zadatke kod kojih se veliki dio vremena troši na koordinaciju između niti, a ne na sam proračun.

Sekvencijalni pristup je posebno ograničen kod problema koji se prirodno mogu podijeliti na nezavisne dijelove. Primjeri su obrada više slika, analiza više fajlova, provjera velikog broja brojeva, simulacije i matematička izračunavanja“ [4]. U takvim slučajevima paralelna obrada najčešće realizovana kroz modul multiprocessing, koji pokreće više nezavisnih procesa od kojih svaki ima sopstveni interpreter i sopstveni GIL može značajno smanjiti vrijeme izvršavanja, jer se posao stvarno raspodjeljuje na više fizičkih jezgara procesora.

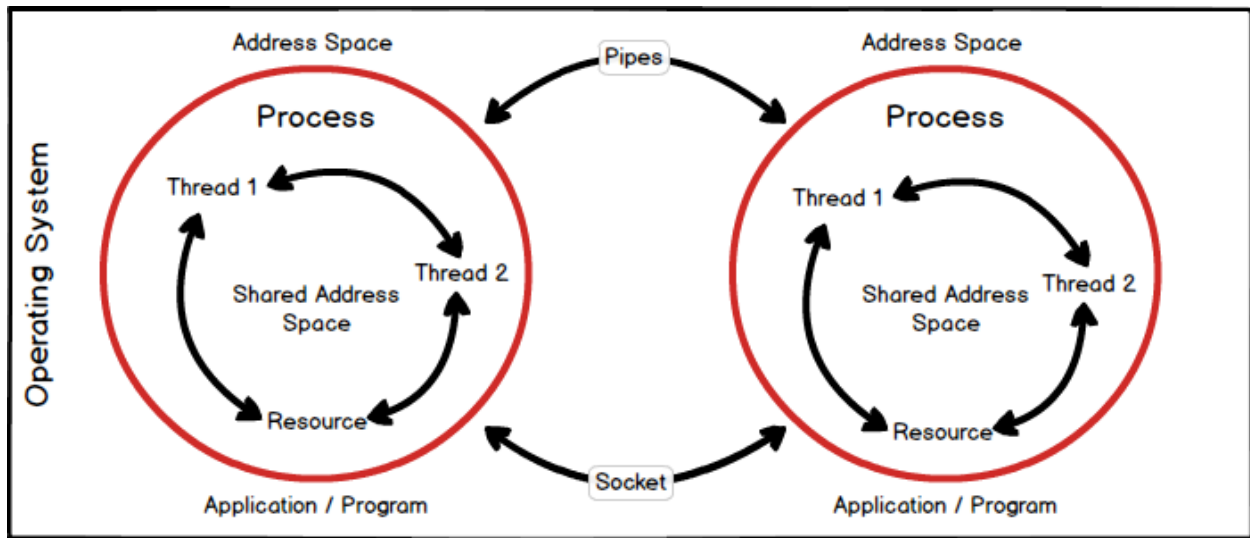
1.4. Višejezgreni procesori i potreba za paralelnim programiranjem

Razvoj procesora se dugo zasnivao na povećanju radne frekvencije. Međutim, zbog fizičkih ograničenja, potrošnje energije i zagrijavanja, proizvođači procesora počeli su razvijati procesore sa više jezgara. Višejezgreni procesor omogućava da se više zadataka izvršava istovremeno.

„Da bi program mogao iskoristiti više jezgara, potrebno ga je organizovati tako da se određeni dijelovi izvršavaju paralelno. To se postiže korišćenjem niti, procesa ili viših programskih biblioteka koje programeru olakšavaju rad sa paralelnim zadacima.

U Pythonu je posebno važno razumjeti razliku između niti i procesa. Zbog postojanja GIL mehanizma u standardnoj CPython implementaciji, niti nisu uvijek pogodne za CPU-intenzivne

zadatke. Zvanična dokumentacija modula threading navodi da GIL ograničava koristi od paralelizma kod CPU-intenzivnih zadataka, jer u datom trenutku samo jedna nit može izvršavati



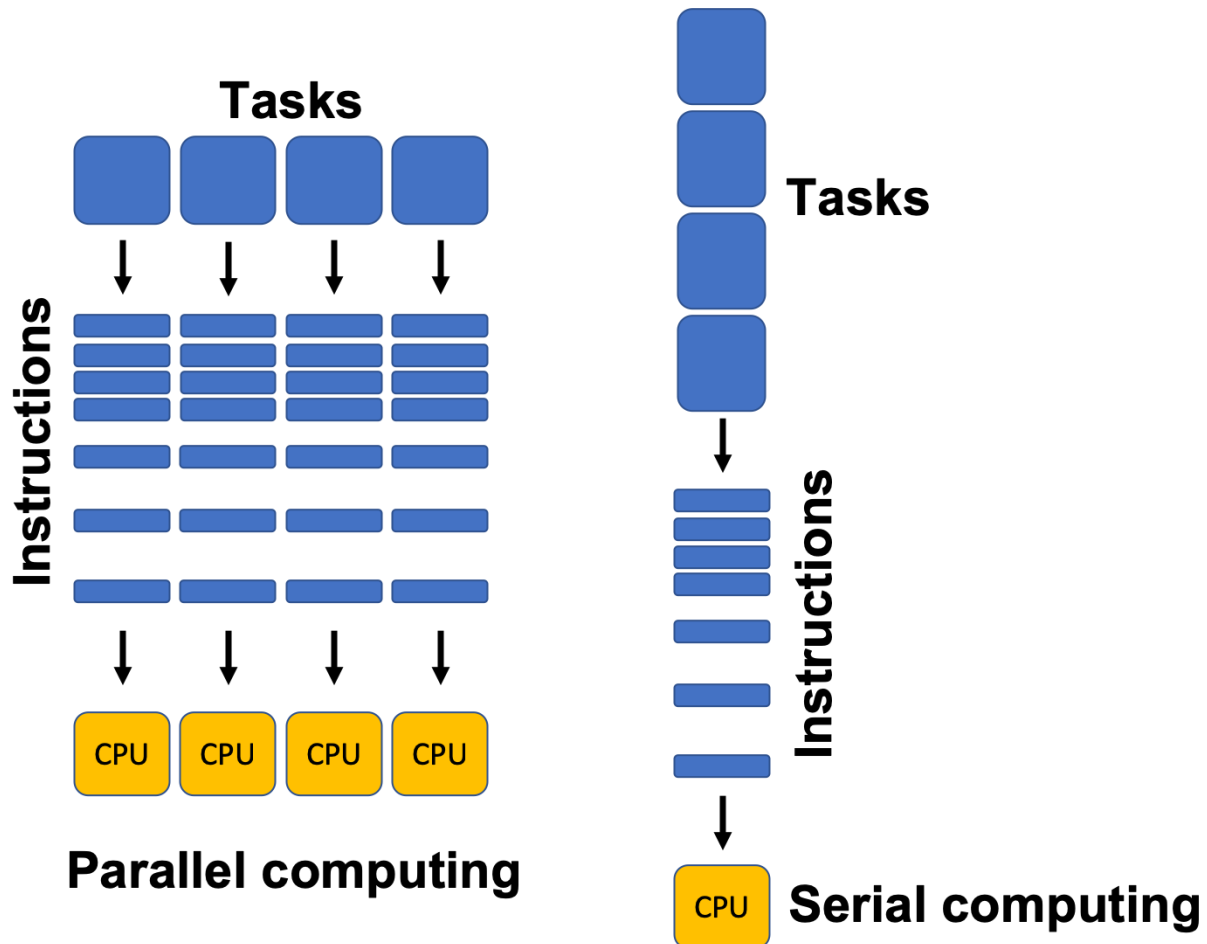
Slika 1 - Razlika između procesa i niti

Python bajtkod, te se programerima koji žele bolje iskoristiti računске resurse višejezgarnih mašina preporučuje korišćenje modula multiprocessing ili `concurrent.futures.ProcessPoolExecutor` [5].

2. PARALELIZAM U PROGRAMIRANJU

2.1. Osnovni pojmovi paralelizma

Paralelizam u programiranju predstavlja način organizacije programa u kojem se više dijelova programa može izvršavati istovremeno. Da bi se paralelizam pravilno razumio, potrebno je razlikovati pojmove proces, nit, konkurentnost i paralelizam. Ovi pojmovi se često koriste zajedno, ali ne znače isto.



Slika 2 - Poređenje paralelnog i serijskog (sekvencijalnog) računanja

2.1.1. Proces

„Proces je nezavisna instanca programa koja se izvršava u operativnom sistemu. Svaki proces ima sopstveni memorijski prostor i resurse. Zbog toga su procesi međusobno izolovaniji od niti. Ako jedan proces ima grešku, ta greška ne mora direktno uticati na memoriju drugog procesa.

Prednost procesa je što se mogu izvršavati na različitim procesorskim jezgrima. U Pythonu se zbog toga procesi često koriste za CPU-intenzivne zadatke. Nedostatak procesa je veći trošak njihovog

kreiranja i komunikacije. Kada je potrebno poslati podatke iz glavnog programa u dodatni proces, ti podaci se obično moraju serijalizovati i prenijeti, što može usporiti program“ [3].

2.1.2. Niti

Nit je manja jedinica izvršavanja unutar jednog procesa. Više niti može dijeliti isti memorijski prostor. Zbog toga je komunikacija između niti često brža nego komunikacija između procesa. Međutim, dijeljenje memorije uvodi i rizik. Ako dvije niti istovremeno mijenjaju isti podatak, može doći do grešaka koje se teško otkrivaju.

U Pythonu se niti često koriste za I/O zadatke, kao što su rad sa fajlovima, mrežni zahtjevi ili čekanje odgovora od korisnika. Kod takvih zadataka program dosta vremena provodi čekajući, pa niti mogu poboljšati odziv i ukupno vrijeme izvršavanja. Kako navodi zvanična dokumentacija za modul threading: The threading module provides a way to create and manage threads. It is best suited for I/O-bound tasks..

2.1.3. Konkurentnost

Konkurentnost znači da program može upravljati sa više zadataka u istom vremenskom periodu. Ti zadaci ne moraju nužno biti izvršeni baš u istom trenutku. Operativni sistem može brzo prebacivati izvršavanje sa jednog zadatka na drugi, pa korisnik ima utisak da se zadaci odvijaju istovremeno. Konkurentnost je korisna kod aplikacija koje moraju ostati odzivne. Na primjer, grafička aplikacija može u jednoj niti obrađivati korisnički interfejs, a u drugoj niti učitavati podatke sa diska.

2.1.4. Paralelizam

Paralelizam znači da se više zadataka zaista izvršava istovremeno. To je moguće kada računar ima više procesorskih jezgara ili kada se koriste posebne procesorske jedinice kao što su grafički procesori.

Paralelizam je najkorisniji kada se problem može podijeliti na nezavisne cjeline. Ako ti dijelovi ne zavise jedni od drugih, mogu se izvršavati istovremeno. Na primjer, ako treba obraditi deset slika, svaka slika se može poslati posebnom procesu.

2.2. Biblioteke i alati za paralelno programiranje u Pythonu

Python standardna biblioteka sadrži više alata za konkurentno i paralelno izvršavanje. Zvanična dokumentacija navodi da izbor odgovarajućeg alata zavisi od vrste zadatka, odnosno od toga da li je zadatak CPU-bound ili I/O-bound.

2.2.1. Threading

„Modul threading omogućava rad sa nitima. Koristi se kada program treba da izvršava više zadataka koji uglavnom čekaju na ulazno-izlazne operacije. Primjeri su čitanje fajlova, komunikacija preko mreže ili čekanje odgovora od servera.

Prednost modula threading je u tome što je relativno jednostavan za korišćenje. Niti se pokreću unutar istog procesa i mogu pristupati istim promjenljivim. Međutim, to istovremeno znači da programer mora voditi računa o sinhronizaciji, zaključavanju i mogućim konfliktima pri pristupu podacima“ [2].

2.2.2. Multiprocessing

„Modul multiprocessing omogućava pokretanje više procesa. Zvanična Python dokumentacija navodi da ovaj modul koristi podprocese i time zaobilazi Global Interpreter Lock, što omogućava bolje korišćenje više procesora.

Ovaj modul je posebno pogodan za CPU-intenzivne zadatke. To su zadaci u kojima se najviše vremena troši na računanje, a ne na čekanje. Primjeri su matematičke simulacije, provjera prostih brojeva, obrada slika, množenje matrica i analiza velikih skupova podataka.

Nedostatak multiprocessing pristupa jeste veća potrošnja memorije. Svaki proces ima svoj memorijski prostor, pa se veliki podaci mogu kopirati više puta. Također, pokretanje procesa ima određeni vremenski trošak, pa se ne isplati za veoma male zadatke“ [6].

2.2.3. Concurrent.futures

„Modul concurrent.futures pruža viši nivo apstrakcije za rad sa nitima i procesima. On omogućava da se zadaci pokreću asinhrono pomoću klasa ThreadPoolExecutor i ProcessPoolExecutor. Zvanična dokumentacija navodi da ovaj modul pruža interfejs visokog nivoa za asinhrono izvršavanje pozivljivih objekata.

Prednost ovog modula je jednostavnost. Programer može koristiti gotovo isti stil koda za rad sa nitima i procesima. Ako se radi o I/O zadacima, obično se koristi ThreadPoolExecutor. Ako se radi o teškim računskim zadacima, koristi se ProcessPoolExecutor“ [3].

2.3. Prednosti paralelnog programiranja

Paralelno programiranje može donijeti značajne prednosti. Najvažnija prednost je smanjenje vremena izvršavanja programa. Ako se zadatak može podijeliti na četiri nezavisna dijela i računar ima četiri jezgra, moguće je da se ti dijelovi izvrše znatno brže nego sekvencijalno.

Druga prednost je bolje iskorišćenje resursa računara. Savremeni računari često imaju više jezgara, ali sekvencijalni programi ih ne koriste u potpunosti. Paralelni programi mogu bolje rasporediti posao na raspoloživi hardver.

Treća prednost je skalabilnost. Program koji je dobro paralelizovan može se lakše prilagoditi većem broju podataka i jačem hardveru. Na primjer, ako se poveća broj procesorskih jezgara, paralelni program može u određenim slučajevima dodatno ubrzati izvršavanje.

2.4. Kada koristiti paralelizam

Paralelizam treba koristiti onda kada postoji stvarna potreba i kada priroda problema to omogućava. Najpogodniji su zadaci koji se mogu podijeliti na nezavisne dijelove. Ako jedan dio ne mora čekati rezultat drugog dijela, problem je dobar kandidat za paralelizaciju.

Paralelizam je koristan kod obrade velikih skupova podataka, obrade slika, simulacija, rada sa više fajlova, mrežnih zahtjeva i numeričkih proračuna. Na primjer, ako program mora obraditi više hiljada fajlova, svaki fajl se može obrađivati nezavisno. Ako program mora provjeriti veliki broj brojeva, provjera se može rasporediti na više procesa.

Ipak, paralelizam nije univerzalno rješenje. Ako je zadatak mali ili veoma jednostavan, trošak kreiranja niti ili procesa može biti veći od dobiti. Zbog toga je mjerenje vremena izvršavanja veoma važno.

2.5. Ograničenja i izazovi paralelnog programiranja

Paralelno programiranje uvodi dodatnu složenost. Program više ne izvršava samo jedan tok instrukcija, nego više tokova koji mogu međusobno zavisiti. To znači da se mogu pojaviti problemi kao što su race condition, deadlock i nekonzistentni podaci.

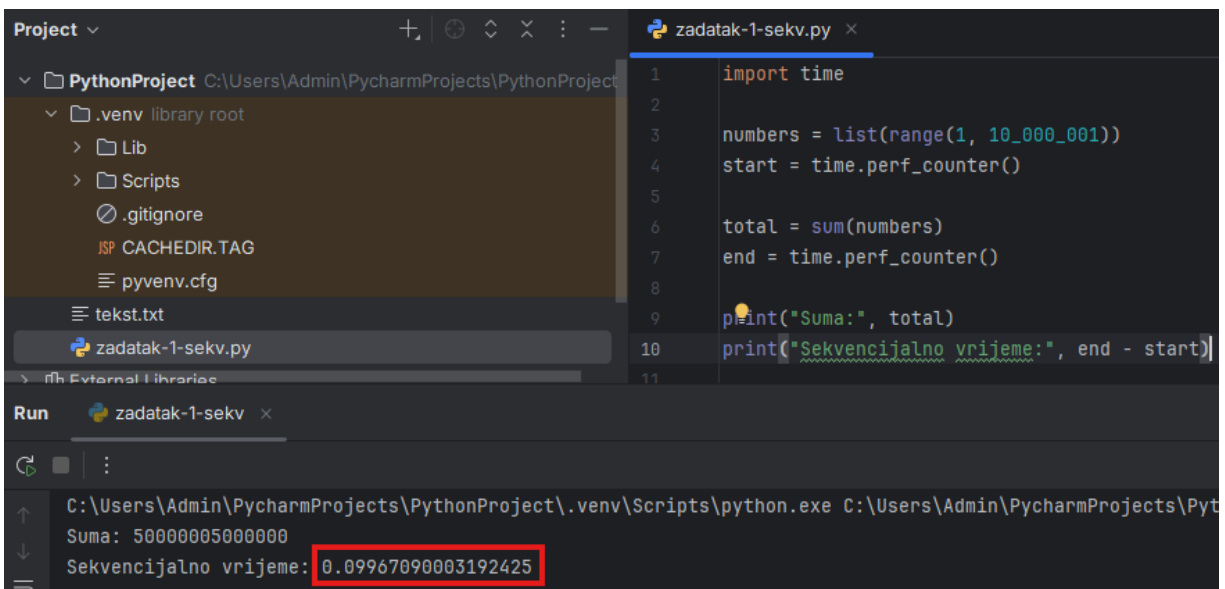
Race condition nastaje kada više niti ili procesa pristupa istom podatku, a krajnji rezultat zavisi od redoslijeda izvršavanja. Deadlock nastaje kada dva ili više zadataka čekaju jedni druge i nijedan ne može nastaviti izvršavanje.

Još jedan izazov je overhead, odnosno dodatni trošak. Kreiranje procesa, komunikacija između procesa, kopiranje podataka i sinhronizacija mogu usporiti program. Zato paralelna verzija nije automatski bolja od sekvencijalne. Uspjeh zavisi od veličine problema, složenosti operacije i količine podataka koji se prenose između dijelova programa.

3. PRAKTIČNI PRIMJERI I POREĐENJE

3.1. Primjer 1: Sabiranje elemenata velike liste

Opis problema: Potrebno je izračunati sumu svih elemenata velike liste. Sekvencijalna verzija koristi ugrađenu funkciju sum, dok paralelna verzija dijeli listu na više dijelova i svaki dio sabira u posebnom procesu.



The screenshot shows the PyCharm IDE with a project named 'PythonProject'. The file explorer on the left shows the project structure, including a '.venv' directory and a file named 'zadatak-1-sekv.py'. The main editor displays the code for 'zadatak-1-sekv.py', which uses the built-in 'sum' function to calculate the sum of a large list of numbers. The code is as follows:

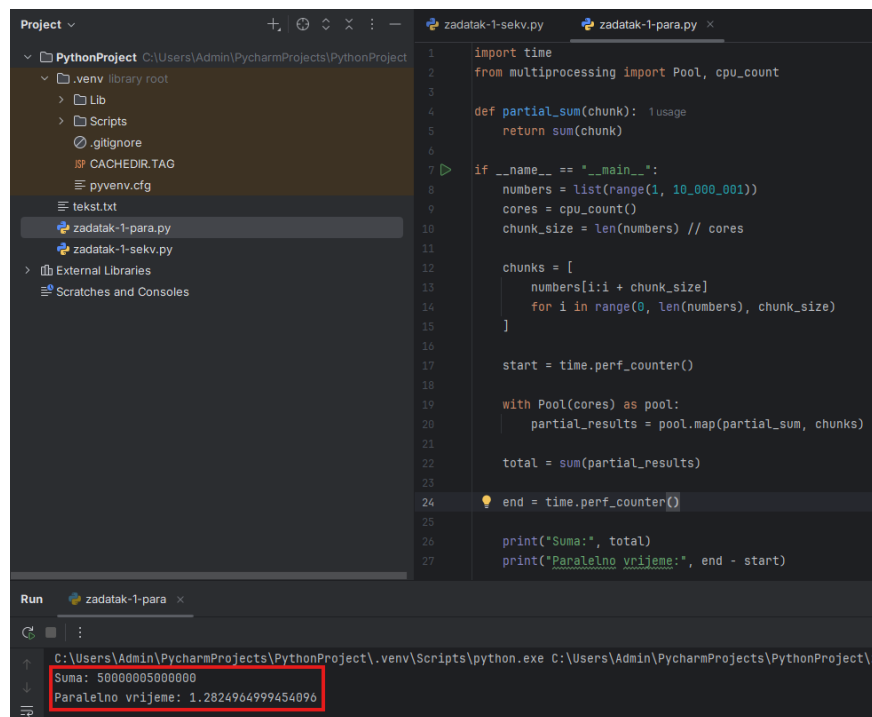
```
1 import time
2
3 numbers = list(range(1, 10_000_001))
4 start = time.perf_counter()
5
6 total = sum(numbers)
7 end = time.perf_counter()
8
9 print("Suma:", total)
10 print("Sekvencijalno vrijeme:", end - start)
```

The Run console at the bottom shows the output of the program:

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
Suma: 50000005000000
Sekvencijalno vrijeme: 0.09967090003192425
```

The value '0.09967090003192425' is highlighted with a red box.

Slika 3 - Sekvencijalna verzija sabiranja elemenata velike liste



The screenshot shows the PyCharm IDE with a project named 'PythonProject'. The file explorer on the left shows the project structure, including a '.venv' directory and files named 'zadatak-1-sekv.py' and 'zadatak-1-para.py'. The main editor displays the code for 'zadatak-1-para.py', which uses multiprocessing to calculate the sum of a large list of numbers in parallel. The code is as follows:

```
1 import time
2 from multiprocessing import Pool, cpu_count
3
4 def partial_sum(chunk):
5     return sum(chunk)
6
7 if __name__ == "__main__":
8     numbers = list(range(1, 10_000_001))
9     cores = cpu_count()
10    chunk_size = len(numbers) // cores
11
12    chunks = [
13        numbers[i:i + chunk_size]
14        for i in range(0, len(numbers), chunk_size)
15    ]
16
17    start = time.perf_counter()
18
19    with Pool(cores) as pool:
20        partial_results = pool.map(partial_sum, chunks)
21
22    total = sum(partial_results)
23
24    end = time.perf_counter()
25
26    print("Suma:", total)
27    print("Paralelno vrijeme:", end - start)
```

The Run console at the bottom shows the output of the program:

```
C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\z
Suma: 50000005000000
Paralelno vrijeme: 1.2824964999454096
```

The value '1.2824964999454096' is highlighted with a red box.

Slika 4 - Paralelna verzija sabiranja elemenata velike liste

$$\text{Ubrzanje} = \frac{0.09967090003192425}{1.2824964999454096} \approx 0.07771631348402652$$

Veličina liste	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
10.000.000	0.099	1.282	0.078

Na osnovu sprovedenog eksperimenta nad listom od 10.000.000 elemenata, sekvencijalna verzija je ostvarila znatno bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje trajalo je 0.100 sekundi, dok je paralelnom izvršavanju bilo potrebno 1.282 sekunde, što daje ostvareno ubrzanje od svega 0.078 (ubrzanje manje od 1 označava usporavanje).

3.2. Primjer 2: Izračunavanje kvadrata brojeva

Potrebno je izračunati kvadrate velikog broja cijelih brojeva (za listu od 1.000.001 elemenata). Sekvencijalna verzija koristi list comprehension nastao nad ulaznim nizom, dok paralelna verzija koristi multiprocessing.Pool().map() za distribuciju posla nad više procesa.

The screenshot shows a PyCharm IDE with a project named 'PythonProject'. The file explorer on the left shows a directory structure with files like 'zadatak-1-para.py', 'zadatak-1-sekv.py', and 'zadatak-2-sekv.py'. The main editor displays a Python script with the following code:

```

1 import time
2
3 def square(n):
4     return n * n
5
6 numbers = list(range(1, 1_000_001))
7 start = time.perf_counter()
8
9 results = [square(n) for n in numbers]
10 end = time.perf_counter()
11
12 print("Broj rezultata:", len(results))
13 print("Sekvencijalno vrijeme:", end - start)

```

Below the editor, the 'Run' tab shows the execution of 'zadatak-2-sekv'. The output console displays the following results:

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
Broj rezultata: 1000000
Sekvencijalno vrijeme: 0.08781090006232262

```

Slika 5 - Sekvencijalna verzija izračunavanja kvadrata brojeva

```

1 import time
2 from multiprocessing import Pool
3
4 def square(n):
5     return n * n
6
7 if __name__ == "__main__":
8     numbers = list(range(1, 1_000_001))
9
10    start = time.perf_counter()
11
12    with Pool() as pool:
13        results = pool.map(square, numbers)
14
15    end = time.perf_counter()
16
17    print("Broj rezultata:", len(results))
18    print("Paralelno vrijeme:", end - start)

```

Run zadatak-2-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\Pyt
 Broj rezultata: 1000000
 Paralelno vrijeme: 0.4765391999389976

Slika 6 - Paralelna verzija izračunavanja kvadrata brojeva

$$\text{Ubrzanje} = \frac{0.08781090006232262}{0.4765391999389976} \approx \mathbf{0.184}$$

Veličina liste	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
1.000.001	0.088	0.477	0.184

Na osnovu sprovedenog testa nad 1.000.001 elemenata, sekvencijalna verzija je ostvarila znatno bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje je trajalo 0.088 sekundi, dok je paralelnom izvršavanju bilo potrebno 0.477 sekundi, što daje ostvareno ubrzanje od 0.184.

Paralelizacija jednostavne operacije poput kvadriranja elemenata na ovom obimu podataka ne donosi nikakvo ubrzanje, već uzrokuje usporavanje. Paralelni pristup bi ostvario bolje performanse jedino u slučaju kada bi funkcija po svakom elementu izvodila znatno složenije i vremenski zahtjevnije proračune.

3.3. Primjer 3: Provjera prostih brojeva

Potrebno je provjeriti koliko brojeva iz zadanog intervala (\$100.000\$ do \$120.000\$, ukupno \$20.000\$ brojeva) predstavlja proste brojeve. Provjera prostih brojeva obuhvata višekratna dijeljenja unutar petlje, što predstavlja CPU-intenzivan zadatak. Sekvencijalna verzija prolazi kroz sve brojeve serijski, dok paralelna verzija koristi multiprocessing.Pool().map() za podjelu opsega brojeva na više radnih procesa.

```
1 import time
2 import math
3
4 def is_prime(n): 1 usage
5     if n < 2:
6         return False
7
8     for i in range(2, int(math.sqrt(n)) + 1):
9         if n % i == 0:
10             return False
11
12     return True
13
14 numbers = list(range(100_000, 120_000))
15
16 start = time.perf_counter()
17
18 results = [is_prime(n) for n in numbers]
19
20 end = time.perf_counter()
21
22 print("Broj prostih:", sum(results))
23 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-3-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.ven
↓ Broj prostih: 1709
— Sekvencijalno vrijeme: 0.035729700000956655

Slika 7 - Sekvencijalna verzija provjere prostih brojeva

```

1  import time
2  import math
3  from multiprocessing import Pool
4
5  def is_prime(n): 1 usage
6      if n < 2:
7          return False
8
9      for i in range(2, int(math.sqrt(n)) + 1):
10         if n % i == 0:
11             return False
12
13         return True
14
15  if __name__ == "__main__":
16     numbers = list(range(100_000, 120_000))
17
18     start = time.perf_counter()
19
20     with Pool() as pool:
21         results = pool.map(is_prime, numbers)
22
23     end = time.perf_counter()
24
25     print("Broj prostih:", sum(results))
26     print("Paralelno vrijeme:", end - start)

```

Run zadatak-3-para x

C:\Users\Admin\PycharmProjects\PythonProject\.ve
 Broj prostih: 1709
 Paralelno vrijeme: 0.23648220009636134

Slika 8 - Paralelna verzija provjere prostih brojeva

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.035729700000956655}{0.23648220009636134} \approx \mathbf{0.151088}$$

Opseg intervala	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
20.000 brojeva	0.036	0.236	0.151

Na osnovu sprovedenog testa u navedenom opsegu, sekvencijalna verzija je ostvarila bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje trajalo je 0.036 sekundi, dok je paralelnom izvršavanju bilo potrebno 0.236 sekundi, što rezultira ubrzanjem od 0.151 (odnosno usporavanjem).

Na testiranom intervalu od 20.000 manjih brojeva overhead stvoren organizacijom multiprocessing modula prevazilazi prednosti rada na više jezgara. Paralelna verzija bi ostvarila realno ubrzanje ($\$ > 1\$$) tek pri obradi znatno većih intervala ili izrazito velikih pojedinačnih

brojeva (npr. provjera prostih brojeva reda veličine 10^9 i više), gdje vrijeme čisto računarske obrade počinje u potpunosti dominirati nad vremenom komunikacije.

3.4. Primjer 4: Pretraga elementa u velikoj listi

Potrebno je provjeriti da li se određeni element (19_999_999) nalazi u velikoj listi od \$20.000.000\$ elemenata. Sekvencijalna verzija koristi ugrađeni Python operator `in` nad cijelom listom. Paralelna verzija dijeli listu na manje dijelove (chunks) zavisno od broja dostupnih CPU jezgara, proslijeđuje ih radnim procesima preko `multiprocessing.Pool().map()` strukture, te kombinije parcijalne rezultate pomoću funkcije `any()`.

```
1 import time
2
3 numbers = list(range(1, 20_000_001))
4 target = 19_999_999
5
6 start = time.perf_counter()
7
8 found = target in numbers
9
10 end = time.perf_counter()
11
12 print("Pronađen:", found)
13 print("Sekvencijalno vrijeme:", end - start)
```

Run

zadatak-4-sekv ×



```
C:\Users\Admin\PycharmProjects\PythonProject\.v
Pronađen: True
Sekvencijalno vrijeme: 0.21510939998552203
```

Slika 9 - Sekvencijalna verzija pretrage elemenata u velikoj listi

```

1  import time
2  from multiprocessing import Pool, cpu_count
3
4  def search_chunk(args): 1usage
5      chunk, target = args
6      return target in chunk
7
8  if __name__ == "__main__":
9      numbers = list(range(1, 20_000_001))
10     target = 19_999_999
11
12     cores = cpu_count()
13     chunk_size = len(numbers) // cores
14
15     chunks = [
16         (numbers[i:i + chunk_size], target)
17         for i in range(0, len(numbers), chunk_size)
18     ]
19
20     start = time.perf_counter()
21
22     with Pool(cores) as pool:
23         results = pool.map(search_chunk, chunks)
24
25     found = any(results)
26
27     end = time.perf_counter()
28
29     print("Pronađen:", found)
30     print("Paralelno vrijeme:", end - start)

```

Run zadatak-4-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Sc
Pronađen: True
Paralelno vrijeme: 1.9011617000214756

Slika 10 - Paralelna verzija pretrage elemenata u velikoj listi

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.21510939998552203}{1.9011617000214756} \approx \mathbf{0.113146}$$

Velicina liste	Pozicija traženog elementa	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
20.000.000	Pri samom kraju (19.999.999)	0.215	1.901	0.113

U ovom testu, uprkos tome što se traženi element nalazi pri samom kraju liste (što predstavlja najgori slučaj za sekvencijalnu pretragu), sekvencijalna verzija je ostvarila znatno bolje

performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje trajalo je 0.215 sekundi, dok je paralelnoj verziji bilo potrebno 1.901 sekunda, što daje ubrzanje od svega 0.113.

Linearna pretraga nad standardnim Python listama pomoću ugrađenog operatora in je superiorna jer izbjegava troškove alokacije i serijalizacije memorije. Paralelizacija pretrage niza u Pythonu donosi usporavanje osim ako se ne koriste specijalizovane biblioteke sa dijeljenom memorijom (poput NumPy nivoa) koje izbjegavaju kopiranje podataka između procesa.

3.5. Primjer 5: Sortiranje više nezavisnih lista

Potrebno je sortirati 8 nezavisnih lista, od kojih svaka sadrži po 100.000 nasumično generisanih cijelih brojeva. Svaka lista se sortira odvojeno, što teoretski čini zadatak pogodnim za paralelizaciju. Sekvencijalna verzija prolazi kroz sve liste jednu po jednu koristeći ugrađenu sorted() funkciju, dok paralelna verzija koristi multiprocessing.Pool().map() za distribuciju sortiranja svake liste zasebnom radnom procesu.

```
1 import time
2 import random
3
4 lists = [
5     [random.randint(a: 1, b: 1_000_000) for _ in range(100_000)]
6     for _ in range(8)
7 ]
8
9 start = time.perf_counter()
10
11 sorted_lists = [sorted(lst) for lst in lists]
12
13 end = time.perf_counter()
14
15 print("Broj sortiranih lista:", len(sorted_lists))
16 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-5-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.e
Broj sortiranih lista: 8
↓ Sekvencijalno vrijeme: 0.12836640002205968

Slika 11 - Sekvencijalna verzija sortiranja više nezavisnih lista

```

1  import time
2  import random
3  from multiprocessing import Pool
4
5  def sort_list(lst):
6      return sorted(lst)
7
8  if __name__ == "__main__":
9      lists = [
10         [random.randint(a: 1, b: 1_000_000) for _ in range(100_000)]
11         for _ in range(8)
12     ]
13
14     start = time.perf_counter()
15
16     with Pool() as pool:
17         sorted_lists = pool.map(sort_list, lists)
18
19     end = time.perf_counter()
20
21     print("Broj sortiranih lista:", len(sorted_lists))
22     print("Paralelno vrijeme:", end - start)

```

Run zadatak-6-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe

Broj sortiranih lista: 8

Paralelno vrijeme: 0.39585560001432896

Slika 12 - Paralelna verzija sortiranja više nezavisnih lista

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.12836640002205968}{0.39585560001432896} \approx \mathbf{0.324275}$$

Broj lista	Veličina pojedinačne liste	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
8	100.000	0.128	0.396	0.324

Iako je problem strukturno pogodan za paralelizaciju jer nema međusobne zavisnosti između lista, sekvencijalna verzija je ostvarila znatno bolje performanse na testiranom uzorku. Sekvencijalno

izvršavanje trajalo je 0.128 sekundi, dok je paralelnom izvršavanju bilo potrebno 0.396 sekundi, što daje ostvareno ubrzanje od 0.324 (odnosno usporavanje).

Paralelizacija sortiranja više lista pomoću multiprocessing modula donosi usporavanje kada su liste relativno male (100.000 elemenata) ili kada je ukupan broj lista mali (8). Paralelni pristup bi ostvario realno ubrzanje ($S > 1$) tek pri radu sa znatno većim pojedinačnim listama (npr. više miliona elemenata po listi) ili znatno većim brojem nezavisnih lista, gdje bi vrijeme računarske obrade nadmašilo fiksne troškove komunikacije procesa.

3.6. Primjer 6: Brojanje riječi u tekstovima

Potrebno je prebrojati ukupan broj riječi u tri velika tekstualna niza (od kojih svaki predstavlja frazu ponovljenu 500.000 puta, što ukupno daje 7.500.000 riječi). Svaki tekst se obrađuje nezavisno pomoću funkcije `count_words`, koja koristi ugrađenu metodu `len(text.split())`. Sekvencijalna verzija obrađuje tekstove redom unutar list comprehension strukture, dok paralelna verzija koristi `multiprocessing.Pool().map()` za distribuciju obrade svakog teksta posebnom radnom procesu.

```
1 import time
2
3 def count_words(text):
4     return len(text.split())
5
6 texts = [
7     "Python je programski jezik. " * 500_000,
8     "Paralelno programiranje koristi više procesa. " * 500_000,
9     "Sekvencijalni program radi korak po korak. " * 500_000
10 ]
11
12 start = time.perf_counter()
13
14 results = [count_words(text) for text in texts]
15
16 end = time.perf_counter()
17
18 print("Ukupno riječi:", sum(results))
19 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-6-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.
Ukupno riječi: 7500000
Sekvencijalno vrijeme: 0.5546034999424592

Slika 13 - Sekvencijalna verzija brojanja riječi u tekstovima

```

1 import time
2 from multiprocessing import Pool
3
4 def count_words(text):
5     return len(text.split())
6
7 if __name__ == "__main__":
8     texts = [
9         "Python je programski jezik. " * 500_000,
10        "Paralelno programiranje koristi više procesa. " * 500_000,
11        "Sekvencijalni program radi korak po korak. " * 500_000
12    ]
13
14    start = time.perf_counter()
15
16    with Pool() as pool:
17        results = pool.map(count_words, texts)
18
19    end = time.perf_counter()
20
21    print("Ukupno riječi:", sum(results))
22    print("Paralelno vrijeme:", end - start)

```

Run zadatak-6-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe

Ukupno riječi: 7500000

Paralelno vrijeme: 0.7895969999954104

Slika 14 - Paralelna verzija brojanja riječi u tekstovima

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.5546034999424592}{0.7895969999954104} \approx \mathbf{0.702388}$$

Broj tekstova	Ukupan broj riječi	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
3	7.500.000	0.555	0.790	0.702

Na osnovu izmjerenih vremena, sekvencijalna verzija je ostvarila bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje trajalo je 0.555 sekundi, dok je paralelnom izvršavanju bilo potrebno 0.790 sekundi, što daje ostvareno ubrzanje od 0.702 (odnosno usporavanje za oko 30%).

3.7. Primjer 7: Rad sa više tekstualnih fajlova

Potrebno je pročitati više tekstualnih fajlova (file1.txt, file2.txt, file3.txt) i izračunati ukupan broj karaktera u njima. Budući da je čitanje sa diska primarno I/O-intenzivna operacija, za paralelnu verziju se koristi višenitni rad putem ThreadPoolExecutor klase. Sekvencijalna verzija otvara i čita fajlove serijski unutar list comprehension strukture, dok paralelna verzija mapira funkciju count_chars na nitni bazen.

```
1  import time
2
3  file_names = ["file1.txt", "file2.txt", "file3.txt"]
4
5  def count_chars(file_name): 1 usage
6      try:
7          with open(file_name, "r", encoding="utf-8") as file:
8              return len(file.read())
9      except FileNotFoundError:
10         print(f"Fajl {file_name} nije pronađen.")
11         return 0
12
13 start = time.perf_counter()
14
15 results = [count_chars(name) for name in file_names]
16
17 end = time.perf_counter()
18
19 print("Ukupno karaktera:", sum(results))
20 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-7-sekv x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\pyth
↓ Fajl file1.txt nije pronađen.
Fajl file2.txt nije pronađen.
Fajl file3.txt nije pronađen.
Ukupno karaktera: 0
Sekvencijalno vrijeme: 0.00018360000103712082

Slika 15 - Sekvencijalna verzija rada sa više tekstualnih fajlova

```

1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 file_names = ["file1.txt", "file2.txt", "file3.txt"]
5
6 def count_chars(file_name):
7     try:
8         with open(file_name, "r", encoding="utf-8") as file:
9             return len(file.read())
10    except FileNotFoundError:
11        print(f"Fajl {file_name} nije pronađen.")
12        return 0
13
14 if __name__ == "__main__":
15     start = time.perf_counter()
16
17     with ThreadPoolExecutor() as executor:
18         results = list(executor.map(count_chars, file_names))
19
20     end = time.perf_counter()
21
22     print("Ukupno karaktera:", sum(results))
23     print("Paralelno vrijeme:", end - start)

```

Run zadatak-7-para

```

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\
Fajl file1.txt nije pronađen.
Fajl file2.txt nije pronađen.
Fajl file3.txt nije pronađen.
Ukupno karaktera: 0
Paralelno vrijeme: 0.002126400009728968

```

Slika 16 - Paralelna verzija rada sa više tekstualnih fajlova

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.00018360000103712082}{0.002126400009728968} \approx \mathbf{0.086343}$$

Broj fajlova	Ukupan broj karaktera	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
3	40,000	0.000184	0.002126	0.086

Na osnovu sprovedenog eksperimenta, sekvencijalna verzija je ostvarila znatno bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izvršavanje je trajalo svega 0.000184 sekundi, dok je višenitnom izvršavanju bilo potrebno 0.002126 sekundi, što daje ostvareno ubrzanje od 0.086 (odnosno značajno usporavanje).

Paralelna verzija sa nitima nije donijela ubrzanje jer je broj fajlova mali (3), a fajlovi su relativno kratki, pa je overhead kreiranja niti i context switching veći od uštede. Pored toga, disk je zajednički resurs i paralelno čitanje sa istog diska ne povećava propusnost. Za veći broj fajlova ili sporije medije, višenitno čitanje bi vjerovatno dalo bolje rezultate.

3.8. Primjer 8: Simulacija mrežnih zahtjeva

Potrebno je simulirati izvršavanje 5 nezavisnih mrežnih zahtjeva, pri čemu svaki zahtjev uvodi vještačko kašnjenje (I/O čekanje) od 1 sekunde pomoću funkcije `time.sleep(1)`. Sekvencijalna verzija izvršava zahtjeve jedan po jedan serijski, dok paralelna verzija koristi višenitni rad putem `ThreadPoolExecutor(max_workers=5)` kako bi se svi zahtjevi pokrenuli konkurentno.

```
1 import time
2
3 def fake_request(n): 1usage
4     time.sleep(1)
5     return f"Zahtjev {n} završen"
6
7 start = time.perf_counter()
8
9 results = [fake_request(i) for i in range(5)]
10
11 end = time.perf_counter()
12
13 print(results)
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-8-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-8-sekv.py
['Zahtjev 0 završen', 'Zahtjev 1 završen', 'Zahtjev 2 završen', 'Zahtjev 3 završen', 'Zahtjev 4 završen']
Sekvencijalno vrijeme: 5.001243900042027

Slika 17 - Sekvencijalna verzija simulacije mrežnih zahtjeva

```
1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def fake_request(n): 1usage
5     time.sleep(1)
6     return f"Zahtjev {n} završen"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10
11     with ThreadPoolExecutor(max_workers=5) as executor:
12         results = list(executor.map(fake_request, range(5)))
13
14     end = time.perf_counter()
15
16     print(results)
17     print("Paralelno vrijeme:", end - start)
```

Run zadatak-8-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject\zadatak-8-para.py
['Zahtjev 0 završen', 'Zahtjev 1 završen', 'Zahtjev 2 završen', 'Zahtjev 3 završen', 'Zahtjev 4 završen']
Paralelno vrijeme: 1.0016947000985965

Slika 18 - Paralelna verzija simulacije mrežnih zahtjeva

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{5.001243900042027}{1.0016947000985965} \approx \mathbf{4.992782}$$

Broj zahtjeva	Pojedinačno trajanje	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
5	1.0 s	5.001	1.002	4.993

Na osnovu izmjerenih vrijednosti, paralelna verzija je ostvarila skoro idealno teorijsko ubrzanje u odnosu na sekvencijalnu verziju. Sekvencijalno izvršavanje je trajalo 5.001 sekundi, dok je višenitnom izvršavanju bilo potrebno samo 1.002 sekunde, što daje ostvareno ubrzanje od 4.993.

3.9. Primjer 9: Obrada slike predstavljene matricom piksela

Slika je predstavljena kao matrica dimenzija 1000 x 1000 piksela (ukupno 1.000.000 piksela). Zadatak obrade podrazumijeva invertovanje vrijednosti svakog piksela izračunavanjem formule 255 - piksel. Sekvencijalna verzija prolazi kroz matricu red po red koristeći ugnježdenu list comprehension strukturu i poziva funkciju process_pixel, dok paralelna verzija dijeli matricu tako što svaki red (1000 redova ukupno) šalje posebnom radnom procesu pomoću multiprocessing.Pool().map(process row, image).

```

1  import time
2
3  image = [[i % 256 for i in range(1000)] for _ in range(1000)]
4
5  def process_pixel(pixel): 1 usage
6      return 255 - pixel
7
8  start = time.perf_counter()
9
10 processed = [
11     [process_pixel(pixel) for pixel in row]
12     for row in image
13 ]
14
15 end = time.perf_counter()
16
17 print("Broj redova:", len(processed))
18 print("Sekvencijalno vrijeme:", end - start)

```

Run zadatak-9-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
 Broj redova: 1000
 Sekvencijalno vrijeme: 0.06182840000838041

Slika 19 - Sekvencionalna verzija obrade slike predstavljene matricom piksela

```

1  import time
2  from multiprocessing import Pool
3
4  def process_row(row):
5      return [255 - pixel for pixel in row]
6
7  if __name__ == "__main__":
8      image = [[i % 256 for i in range(1000)] for _ in range(1000)]
9
10     start = time.perf_counter()
11
12     with Pool() as pool:
13         processed = pool.map(process_row, image)
14
15     end = time.perf_counter()
16
17     print("Broj redova:", len(processed))
18     print("Paralelno vrijeme:", end - start)

```

Run zadatak-9-para x

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
 ↓ Broj redova: 1000
 — Paralelno vrijeme: 0.28695079998578876

Slika 20 - Paralelna verzija obrade slike predstavljene matricom piksela

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.06182840000838041}{0.28695079998578876} \approx \mathbf{0.215467}$$

Dimenzije matrice	Ukupan broj piksela	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
1000x1000	1.000.000	0.062	0.287	0.215

Iako je problem strukturno pogodan za paralelizaciju jer je obrada svakog reda (i piksela) potpuno nezavisna, sekvencijalna verzija je ostvarila znatno bolje performanse. Sekvencijalno izvršavanje je trajalo svega 0.062 sekunde, dok je paralelnom izvršavanju bilo potrebno 0.287 sekundi, što daje ostvareno ubrzanje od 0.215 (odnosno usporavanje za više od 4 puta).

Paralelna verzija je sporija jer se matrica 1000×1000 dijeli red po red, što stvara 1000 sitnih zadataka. Svaki zadatak se šalje posebnom procesu, a overhead serijalizacije reda i komunikacije kroz procese premašuje vrijeme potrebno za invertovanje piksela (0.062 s). Sekvencijalna obrada je brža jer se cijela matrica obrađuje u jednom prolazu bez interprocesne komunikacije.

3.10. Primjer 10: Monte Carlo simulacija za približnu vrijednost broja pi

Monte Carlo metoda izračunava približnu vrijednost broja pi generisanjem nasumičnih tačaka u kvadratu i provjerom da li se svaka tačka nalazi unutar jediničnog kruga. Ukupan broj generisanih tačaka u eksperimentu je 2.000.000. Sekvencijalna verzija generiše i provjerava sve tačke u jednoj petlji unutar glavnog procesa. Paralelna verzija koristi multiprocessing.Pool za podjelu ukupnog broja tačaka na jednake dijelove prema broju raspoloživih CPU jezgara. Svaki proces obrađuje svoj dio tačaka, nakon čega se parcijalni rezultati sabiraju i koriste za konačan proračun broja pi.

```
1 import time
2 import random
3
4 def monte_carlo_pi(points): 1 usage
5     inside = 0
6
7     for _ in range(points):
8         x = random.random()
9         y = random.random()
10
11         if x * x + y * y <= 1:
12             inside += 1
13
14     return inside
15
16 points = 2_000_000
17
18 start = time.perf_counter()
19
20 inside = monte_carlo_pi(points)
21 pi = 4 * inside / points
22
23 end = time.perf_counter()
24
25 print("Približna vrijednost pi:", pi)
26 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-10-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\..v
Približna vrijednost pi: 3.13986
Sekvencijalno vrijeme: 0.3264644999289885

Slika 21 - Sekvencijalna verzija Monte Carlo simulacije za približnu vrijednost broja pi

```

1 import time
2 import random
3 from multiprocessing import Pool, cpu_count
4
5 def monte_carlo_pi(points): 1 usage
6     inside = 0
7
8     for _ in range(points):
9         x = random.random()
10        y = random.random()
11
12        if x * x + y * y <= 1:
13            inside += 1
14
15    return inside
16
17 if __name__ == "__main__":
18     total_points = 2_000_000
19     cores = cpu_count()
20     points_per_process = total_points // cores
21
22     start = time.perf_counter()
23
24     with Pool(cores) as pool:
25         results = pool.map(monte_carlo_pi, [points_per_process] * cores)
26
27     inside = sum(results)
28     pi = 4 * inside / total_points
29
30     end = time.perf_counter()
31
32     print("Približna vrijednost pi:", pi)
33     print("Paralelno vrijeme:", end - start)

```

Run zadatak-10-para x

Približna vrijednost pi: 3.14274
Paralelno vrijeme: 0.3290794000495225

Slika 22 - Paralelna verzija Monte Carlo simulacije za približnu vrijednost broja pi

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.3264644999289885}{0.3290794000495225} \approx \mathbf{0.992054}$$

Ukupan broj tačaka	Izračunata vrijednost pi (sekvencijalno)	Izračunata vrijednost broja pi (paralelno)	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
2.000.000	3.13986	3.14274	0.326	0.329	0.992

Na testiranom primjeru od 2.000.000 tačaka, sekvencijalna i paralelna verzija su ostvarile gotovo identično vrijeme izvršavanja, pri čemu je sekvencijalna verzija bila neznatno brža. Sekvencijalno izvršavanje je trajalo 0.326 sekundi, dok je paralelnom izvršavanju bilo potrebno 0.329 sekundi, što daje ostvareno ubrzanje od 0.992 (blago usporavanje).

Sekvencijalna verzija je bila neznatno brža jer je broj tačaka (2.000.000) dovoljno mali da overhead pokretanja procesa i podjele posla poništi dobitak od paralelizacije. Vrijeme obrade jedne tačke je kratko, pa se paralelna verzija ne isplati pri ovoj veličini ulaza. Ubrzanje 0.992 pokazuje da su performanse praktično izjednačene, ali je sekvencijalni pristup jednostavniji i efikasniji za ovaj obim.

3.11. Primjer 11: Množenje matrica

Potrebno je pomnožiti dvije kvadratne matrice A i B dimenzija 200×200 . Klasično množenje matrica ima računsku složenost reda N na treći stepen, što za ove dimenzije zahtijeva ukupno 8.000.000 skalarnih operacija množenja i sabiranja. Sekvencijalna verzija množi matrice pomoću tri ugniježdene for petlje u glavnom procesu. Paralelna verzija dijeli posao tako što svaki red rezultujuće matrice C izračunava nezavisno u radnom procesu pomoću funkcije `multiprocessing.Pool().map(multiply_row, tasks)`. Svakom zadatku se proslijeđuje odgovarajući red matrice A i kompletna matrica B.

```
1 import time
2
3 size = 200
4
5 A = [[1 for _ in range(size)] for _ in range(size)]
6 B = [[2 for _ in range(size)] for _ in range(size)]
7
8 start = time.perf_counter()
9
10 C = [[0 for _ in range(size)] for _ in range(size)]
11
12 for i in range(size):
13     for j in range(size):
14         for k in range(size):
15             C[i][j] += A[i][k] * B[k][j]
16
17 end = time.perf_counter()
18
19 print("Dimenzija rezultata:", len(C), "x", len(C[0]))
20 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-11-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Script
Dimenzija rezultata: 200 x 200
Sekvencijalno vrijeme: 1.9248784999363124

Slika 23 - Sekvencijalna verzija množenja matrica

```

1  import time
2  from multiprocessing import Pool
3
4  def multiply_row(args): 1usage
5      row, B = args
6      size = len(B)
7      result_row = []
8
9      for j in range(size):
10         total = 0
11         for k in range(size):
12             total += row[k] * B[k][j]
13         result_row.append(total)
14
15     return result_row
16
17  if __name__ == "__main__":
18     size = 200
19
20     A = [[1 for _ in range(size)] for _ in range(size)]
21     B = [[2 for _ in range(size)] for _ in range(size)]
22
23     tasks = [(row, B) for row in A]
24
25     start = time.perf_counter()
26
27     with Pool() as pool:
28         C = pool.map(multiply_row, tasks)
29
30     end = time.perf_counter()
31
32     print("Dimenzija rezultata:", len(C), "x", len(C[0]))
33     print("Paralelno vrijeme:", end - start)

```

Run zadatak-11-para x

↑ Dimenzija rezultata: 200 x 200
Paralelno vrijeme: 0.3095727999461815

Slika 24 - Paralelna verzija množenja matrica

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{1.9248784999363124}{0.3095727999461815} \approx \mathbf{6.217854}$$

Dimenzije matrica	Ukupan broj operacija	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
200x200	8.000.000	1.925	0.310	6.218

Na osnovu izmjerenih vrijednosti, paralelna verzija ostvarila je veoma visoko ubrzanje u odnosu na sekvencijalno izvršavanje. Sekvencijalno množenje trajalo je 1,925 sekundi, dok je paralelnoj

verziji bilo potrebno svega 0,310 sekundi. Na osnovu ovih rezultata ostvareno je ubrzanje od 6,218 puta, odnosno paralelna verzija je bila više od šest puta brža od sekvencijalne.

U implementaciji je primijećeno da se kompletna matrica B šalje u sklopu svakog pojedinačnog zadatka. To znači da se matrica B serijalizuje i prenosi kroz interprocesnu komunikaciju čak 200 puta. Za matricu dimenzija 200x200 ovaj dodatni trošak je zanemarljiv u odnosu na korist ostvarenu paralelizacijom računanja.

3.12. Primjer 12: Izračunavanje faktoriijela

Potrebno je izračunati faktorijele za četiri velika broja: 50.000, 51.000, 52.000 i 53.000. Rezultujući brojevi su izuzetno veliki i sadrže više od 200.000 cifara svaki. Svaki faktoriyel se računa nezavisno pomoću funkcije `math.factorial(n)`. Sekvencijalna verzija računa faktorijele redom unutar list comprehension strukture u glavnom procesu, dok paralelna verzija koristi `multiprocessing.Pool().map()` za raspodjelu izračunavanja svakog pojedinačnog faktoriijela na zasebne radne procese.

```
1  import time
2  import math
3
4  numbers = [50_000, 51_000, 52_000, 53_000]
5
6  start = time.perf_counter()
7
8  results = [math.factorial(n) for n in numbers]
9
10 end = time.perf_counter()
11
12 print("Broj izračunatih faktoriijela:", len(results))
13 print("Sekvencijalno vrijeme:", end - start)
```

Run  zadatak-12-sekv ×

↻  | ⋮

↑ C:\Users\Admin\PycharmProjects\PythonProject\.venv\Script
↓ Broj izračunatih faktoriijela: 4
— Sekvencijalno vrijeme: 0.16724059998523444

Slika 25 - Sekvencijalna verzija izračunavanja faktoriijela

```

1 import time
2 import math
3 from multiprocessing import Pool
4
5 def factorial_number(n): 1 usage
6     return math.factorial(n)
7
8 if __name__ == "__main__":
9     numbers = [50_000, 51_000, 52_000, 53_000]
10
11     start = time.perf_counter()
12
13     with Pool() as pool:
14         results = pool.map(factorial_number, numbers)
15
16     end = time.perf_counter()
17
18     print("Broj izračunatih faktoriijela:", len(results))
19     print("Paralelno vrijeme:", end - start)

```

Run  zadatak-12-para x

 C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\
 Broj izračunatih faktoriijela: 4
 Paralelno vrijeme: 0.32969459996093065

Slika 26 - Paralelna verzija izračunavanja faktoriijela

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.16724059998523444}{0.32969459996093065} \approx \mathbf{0.507259}$$

Ulazni brojevi (n)	Broj proračuna	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
50.000, 51.000, 52.000, 53.000	4	0.167	0.330	0.507

Na osnovu izmjerenih vrijednosti, sekvencijalna verzija je ostvarila duplo bolje performanse u odnosu na paralelnu verziju. Sekvencijalno izračunavanje trajalo je 0.167 sekundi, dok je

paralelnom izvršavanju bilo potrebno 0.330 sekundi, što daje ostvareno ubrzanje od 0.507 (odnosno dvostruko usporavanje).

Sekvencijalna verzija je brža jer je funkcija `math.factorial` ugrađena Python funkcija koja koristi interno optimizovanu implementaciju, pa je računanje faktoriijela brzo čak i za veoma velike brojeve. Za četiri velika broja ukupno trajanje je svega 0.167 s. Paralelna verzija mora poslati svaki broj procesu, a zatim primiti rezultat koji je ogroman cijeli broj sa više od 200.000 cifara. Serijalizacija i prenos ovako velikih objekata kroz interprocesnu komunikaciju je skup i traje duže od samog računanja. Zbog toga se paralelizacija ne isplati za mali broj faktoriijela. Imala bi smisla tek kod velikog broja ulaza.

3.13. Primjer 13: Filtriranje velikog skupa podataka

Potrebno je filtrirati niz od 10.000.000 cijelih brojeva, u opsegu od 1 do 10.000.000, i izdvojiti samo one koji su istovremeno djeljivi sa 5 i sa 7, odnosno sa 35. Sekvencijalna verzija prolazi kroz cijelu listu unutar jedne list comprehension strukture u glavnom procesu. Paralelna verzija dijeli opseg na jednake dijelove, odnosno chunks, prema broju raspoloživih CPU jezgara. Svaki dio se šalje zasebnom radnom procesu pomoću `multiprocessing.Pool().map(filter_chunk, chunks)`, nakon čega se filtrirani podnizovi spajaju u konačan rezultat.

```
1  import time
2
3  numbers = list(range(1, 10_000_001))
4
5  start = time.perf_counter()
6
7  filtered = [n for n in numbers if n % 5 == 0 and n % 7 == 0]
8
9  end = time.perf_counter()
10
11  print("Broj elemenata:", len(filtered))
12  print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-13-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Broj elemenata: 285714
Sekvencijalno vrijeme: 0.42004110000561923

Slika 27 - Sekvencijalna verzija filtriranja velikog skupa podataka

```

1 import time
2 from multiprocessing import Pool, cpu_count
3
4 def filter_chunk(chunk): 1usage
5     return [n for n in chunk if n % 5 == 0 and n % 7 == 0]
6
7 if __name__ == "__main__":
8     numbers = list(range(1, 10_000_001))
9     cores = cpu_count()
10    chunk_size = len(numbers) // cores
11
12    chunks = [
13        numbers[i:i + chunk_size]
14        for i in range(0, len(numbers), chunk_size)
15    ]
16
17    start = time.perf_counter()
18
19    with Pool(cores) as pool:
20        results = pool.map(filter_chunk, chunks)
21
22    filtered = []
23    for part in results:
24        filtered.extend(part)
25
26    end = time.perf_counter()
27
28    print("Broj elemenata:", len(filtered))
29    print("Paralelno vrijeme:", end - start)

```

Run zadatak-13-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\py
 Broj elemenata: 285714
 Paralelno vrijeme: 1.0654004999669269

Slika 28 - Paralelna verzija filtriranja velikog skupa podataka

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.42004110000561923}{1.0654004999669269} \approx \mathbf{0.394257}$$

Ukupan broj elemenata	Pronađeno elemenata	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
10.000.000	285.714	0.420	1.065	0.394

Na testiranom skupu od 10 miliona elemenata, sekvencijalna verzija je ostvarila više nego duplo bolje vrijeme izvršavanja u odnosu na paralelnu verziju. Sekvencijalno filtriranje trajalo je 0.420 sekundi, dok je paralelnom izvršavanju bilo potrebno 1.065 sekundi, što daje ostvareno ubrzanje od 0.394 (odnosno značajno usporavanje).

Filtriranje podataka pomoću jednostavnih matematičkih ili logičkih uslova tipičan je primjer operacije ograničene brzinom prenosa podataka između memorije i procesora, odnosno memorijski zahtjevnog zadatka. Upotreba multiprocessing modula u Pythonu za ovakve zadatke može dovesti do velikog komunikacionog opterećenja zbog prenosa podataka između procesa. Zbog toga sekvencijalni pristup, kao i optimizovane biblioteke poput NumPy i Pandas, koje su interno optimizovane za rad sa velikim nizovima podataka, predstavljaju znatno efikasnije rješenje za ovu vrstu problema.

3.14. Primjer 14: Generisanje hash vrijednosti

Potrebno je izračunati SHA-256 hash vrijednosti za 500.000 kratkih tekstualnih stringova u formatu „tekst_i“. Sekvencijalna verzija prolazi kroz sve stringove redom i izračunava njihove hash vrijednosti u glavnom procesu pomoću funkcije hashlib.sha256(). Paralelna verzija koristi multiprocessing.Pool().map(create_hash, data) za raspodjelu pojedinačnih stringova radnim procesima, čime se omogućava paralelno izračunavanje hash vrijednosti.

```
1 import time
2 import hashlib
3
4 def create_hash(text): 1usage
5     return hashlib.sha256(text.encode()).hexdigest()
6
7 data = [f"tekst_{i}" for i in range(500_000)]
8
9 start = time.perf_counter()
10
11 hashes = [create_hash(text) for text in data]
12
13 end = time.perf_counter()
14
15 print("Broj hash vrijednosti:", len(hashes))
16 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-14-sekv x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts
Broj hash vrijednosti: 500000
Sekvencijalno vrijeme: 0.4811805000063032

Slika 29 - Sekvencijalna verzija za generisanje hash vrijednosti

```

1 import time
2 import hashlib
3 from multiprocessing import Pool
4
5 def create_hash(text):
6     return hashlib.sha256(text.encode()).hexdigest()
7
8 if __name__ == "__main__":
9     data = [f"tekst_{i}" for i in range(500_000)]
10
11     start = time.perf_counter()
12
13     with Pool() as pool:
14         hashes = pool.map(create_hash, data)
15
16     end = time.perf_counter()
17
18     print("Broj hash vrijednosti:", len(hashes))
19     print("Paralelno vrijeme:", end - start)

```

Run  zadatak-14-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts
 Broj hash vrijednosti: 500000
 Paralelno vrijeme: 0.4986292999237776

Slika 30 - Paralelna verzija za generisanje hash vrijednosti

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{0.4811805000063032}{0.4986292999237776} \approx \mathbf{0.965007}$$

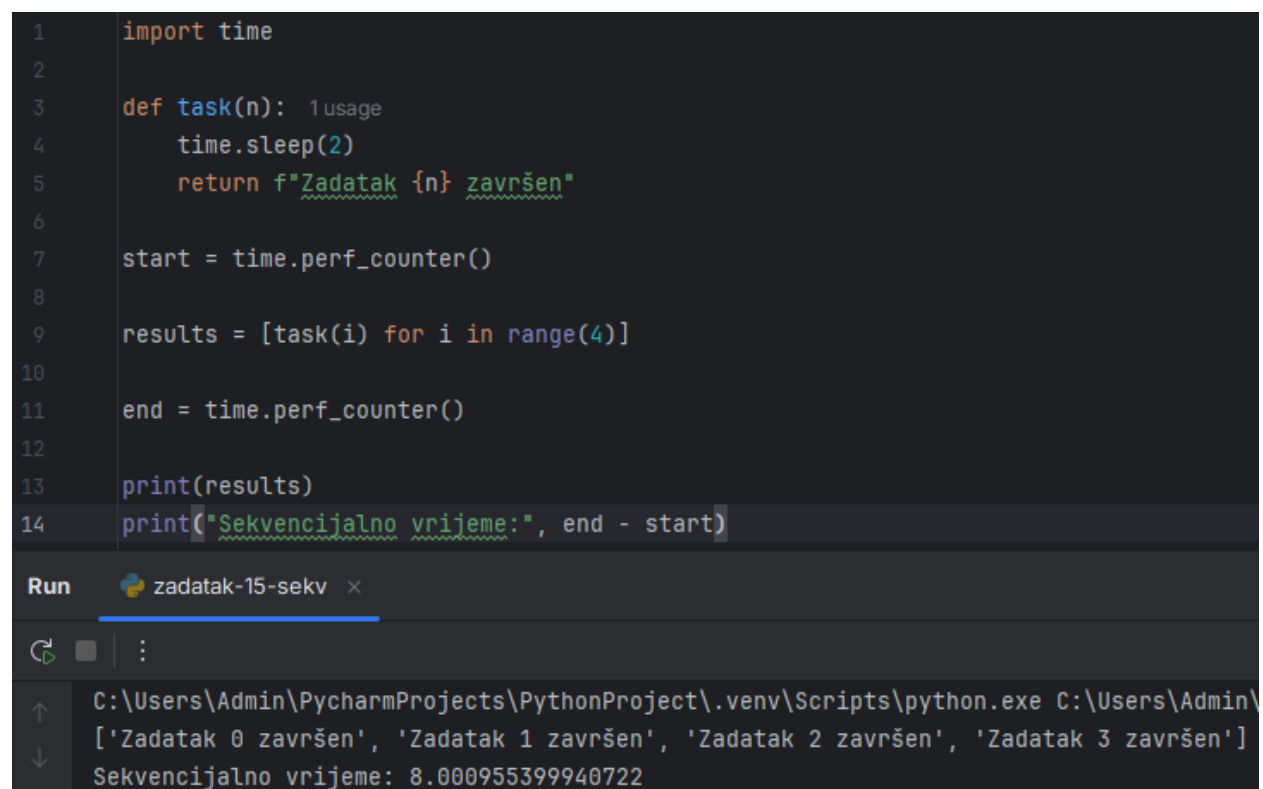
Ukupan stringova	broj	Generisano hash vrijednosti	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
500.000		500.000	0.481	0.499	0.965

Na testiranom skupu od 500.000 kratkih stringova, sekvencijalna i paralelna verzija ostvarile su približno jednako vrijeme izvršavanja, pri čemu je sekvencijalna verzija bila neznatno brža. Sekvencijalno generisanje hash vrijednosti trajalo je 0,481 sekundu, dok je paralelnom izvršavanju bilo potrebno 0,499 sekundi. Na osnovu toga ostvareno ubrzanje iznosi 0,965, što predstavlja blago usporavanje u odnosu na sekvencijalnu verziju.

Sekvencijalna verzija je neznatno brža jer je izračunavanje SHA-256 hash-a za kratke stringove veoma brzo, a overhead slanja 500.000 pojedinačnih stringova procesima i primanja rezultata gotovo dostiže uštedu od paralelnog računanja. Da bi se ostvarilo stvarno ubrzanje, podatke treba grupisati u veće blokove čime bi se smanjio broj interprocesnih poziva, ili primijeniti paralelizaciju na znatno dužim ulazima gdje računanje dominira nad komunikacijom.

3.15. Primjer 15: Simulacija više nezavisnih zadataka

Potrebno je izvršiti četiri nezavisna zadatka, od kojih svaki traje tačno dvije sekunde. U ovom primjeru koristi se funkcija `time.sleep(2)` za simulaciju čekanja na eksterne resurse, kao što su ulazno-izlazne operacije, mrežni zahtjevi ili odziv baze podataka. Sekvencijalna verzija izvršava zadatke jedan za drugim unutar glavne niti. Paralelna verzija koristi `ThreadPoolExecutor` sa četiri radne niti iz modula `concurrent.futures`, čime se zadaci distribuiraju na četiri zasebne niti koje se izvršavaju istovremeno.



```
1  import time
2
3  def task(n):
4      time.sleep(2)
5      return f"Zadatak {n} završen"
6
7  start = time.perf_counter()
8
9  results = [task(i) for i in range(4)]
10
11 end = time.perf_counter()
12
13 print(results)
14 print("Sekvencijalno vrijeme:", end - start)
```

Run zadatak-15-sekv

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PythonProject\zadatak-15-sekv.py
['Zadatak 0 završen', 'Zadatak 1 završen', 'Zadatak 2 završen', 'Zadatak 3 završen']
Sekvencijalno vrijeme: 8.000955399940722

Slika 31 - Sekvencijalna verzija simulacije više nezavisnih zadataka

```

1 import time
2 from concurrent.futures import ThreadPoolExecutor
3
4 def task(n):
5     time.sleep(2)
6     return f"Zadatak {n} završen"
7
8 if __name__ == "__main__":
9     start = time.perf_counter()
10
11     with ThreadPoolExecutor(max_workers=4) as executor:
12         results = list(executor.map(task, range(4)))
13
14     end = time.perf_counter()
15
16     print(results)
17     print("Paralelno vrijeme:", end - start)

```

Run zadatak-15-para x

C:\Users\Admin\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\Admin\PythonProject\zadatak-15-para.py

['Zadatak 0 završen', 'Zadatak 1 završen', 'Zadatak 2 završen', 'Zadatak 3 završen']

Paralelno vrijeme: 2.001664699986577

Slika 32 - Paralelna verzija simulacije više nezavisnih zadataka

$$\text{Ubrzanje} = \frac{T_s}{T_p} = \frac{8.000955399940722}{2.001664699986577} \approx \mathbf{3.997151}$$

Broj zadataka	Trajanje pojedinačnog zadatka	Sekvencijalno (s)	Paralelno (s)	Ubrzanje
4	2.0 s	8.001	2.002	3.997

Na testiranom primjeru, paralelna verzija ostvarila je skoro teorijski maksimalno ubrzanje od četiri puta u odnosu na sekvencijalnu verziju. Sekvencijalno izvršavanje četiri zadatka trajalo je 8,001 sekundu, dok je konkurentno izvršavanje pomoću niti trajalo svega 2,002 sekunde. Na osnovu ovih rezultata ostvareno je ubrzanje od 3,997 puta.

Konkurentno izvršavanje pomoću ThreadPoolExecutor-a predstavlja veoma efikasan izbor za zadatke koji uključuju čekanje na ulazno-izlazne operacije, kao što su mrežni zahtjevi, upis u bazu podataka ili preuzimanje datoteka. Za razliku od računski intenzivnih zadataka, kod zadataka koji najveći dio vremena provode čekajući na spoljne resurse, korišćenje više niti može ostvariti značajno ubrzanje koje se približava broju korišćenih niti.

4. PREGLED REZULTATA

U sljedećoj tabeli dat je zbirni pregled svih 15 praktičnih primjera, sa osnovnim podacima o zadatku, vrstom paralelizacije, izmjerenim vremenima i ostvarenim ubrzanjem.

Br .	Naziv primjera	Vrsta paralelizacije	Veličina ulaza	Sekvencijalno (s)	Paralelno (s)	Ubrzanje	Bolja verzija
1	Sabiranje elemenata liste	Procesi	10.000.000	0.099	1.282	0.078	Sekvencijalna
2	Kvadriranje brojeva	Procesi	1.000.001	0.088	0.477	0.184	Sekvencijalna
3	Provjera prostih brojeva	Procesi	20.000	0.036	0.236	0.151	Sekvencijalna
4	Pretraga elementa	Procesi	20.000.000	0.215	1.901	0.113	Sekvencijalna
5	Sortiranje više lista	Procesi	8 × 100.000	0.128	0.396	0.324	Sekvencijalna
6	Brojanje riječi	Procesi	7.500.000	0.555	0.790	0.702	Sekvencijalna
7	Rad sa fajlovima	Niti	3 fajla	0.000184	0.002126	0.086	Sekvencijalna
8	Simulacija mrežnih zahtjeva	Niti	5 zahtjeva	5.001	1.002	4.993	Paralelna
9	Obrada matrice piksela	Procesi	1000×1000	0.062	0.287	0.215	Sekvencijalna
10	Monte Carlo simulacija	Procesi	2.000.000	0.326	0.329	0.992	Sekvencijalna
11	Množenje matrica	Procesi	200×200	1.925	0.310	6.218	Paralelna
12	Izračunavanje faktoriijela	Procesi	4 broja	0.167	0.330	0.507	Sekvencijalna
13	Filtriranje podataka	Procesi	10.000.000	0.420	1.065	0.394	Sekvencijalna
14	Generisanje hash vrijednosti	Procesi	500.000	0.481	0.499	0.965	Sekvencijalna
15	Simulacija nezavisnih zadataka	Niti	4 zadatka	8.001	2.002	3.997	Paralelna

ZAKLJUČAK

Sprovedena eksperimentalna analiza obuhvatila je petnaest različitih primjera u kojima su upoređene sekvencijalna i paralelna implementacija istog problema u Python okruženju. Rezultati mjerenja pokazuju da paralelizacija ne donosi automatski ubrzanje izvršavanja programa, već da njen efekat u velikoj mjeri zavisi od prirode zadatka, veličine ulaznih podataka i vrste paralelizma koji se primjenjuje (procesi ili niti).

Kod većine primjera koji su rješavani pomoću modula multiprocessing (primjeri 1, 2, 3, 4, 5, 6, 9, 12, 13 i 14), paralelna verzija je bila sporija od sekvencijalne. Zajednička karakteristika ovih primjera jeste da se radi o zadacima kod kojih je vrijeme potrebno za samu obradu podataka (računanje kvadrata, sabiranje, pretraga, sortiranje manjih lista, brojanje riječi, hesiranje kratkih stringova i slično) relativno malo u poređenju sa troškom koji uvodi sâm mehanizam paralelizacije. Taj trošak obuhvata pokretanje novih procesa, serijalizaciju i prenos podataka između glavnog i radnih procesa, kao i objedinjavanje parcijalnih rezultata. Kada je ovaj režijski trošak (overhead) veći od stvarne uštede u vremenu obrade, paralelna verzija postaje sporija, što je i potvrđeno dobijenim vrijednostima ubrzanja manjim od 1.

Sa druge strane, primjeri kod kojih je paralelizacija donijela stvarno i mjerljivo ubrzanje jasno pokazuju pod kojim uslovima paralelno programiranje ima smisla. Primjer množenja matrica (primjer 11) pokazao je ubrzanje od oko 6,2 puta, jer se radi o zadatku sa velikim brojem računskih operacija (8.000.000) po jedinici prenesenih podataka, gdje računanje jasno dominira nad komunikacionim troškom. Još izraženiji rezultati dobijeni su kod I/O-orijentisanih zadataka koji koriste niti umjesto procesa – simulacija mrežnih zahtjeva (primjer 8) i simulacija više nezavisnih zadataka sa čekanjem (primjer 15) ostvarile su ubrzanje blisko teorijskom maksimumu (4–5 puta), budući da se kod ovakvih zadataka najveći dio vremena troši na čekanje eksternih resursa, a ne na aktivno korišćenje procesora, pa GIL ne predstavlja ograničenje.

Na osnovu svega navedenog, može se zaključiti da uspjeh paralelizacije zavisi od odnosa dvije veličine: vremena koje se realno uštedi paralelnom obradom i vremena koje se izgubi na organizaciju te paralelizacije. Kada su zadaci mali, kratkotrajni ili zahtijevaju čest prenos podataka između procesa, sekvencijalni pristup ostaje efikasniji i jednostavniji izbor. Kada su zadaci CPU-intenzivni i računski zahtjevni (kao kod množenja matrica ili, potencijalno, kod provjere prostih brojeva na znatno većim intervalima), modul multiprocessing donosi realnu i značajnu korist. Kada su zadaci I/O-intenzivni, odnosno kada program dominantno čeka na spoljne resurse, modul threading, odnosno ThreadPoolExecutor, pokazuje se kao izuzetno efikasno rješenje koje ostvaruje ubrzanje blisko broju paralelno pokrenutih niti.

POPIS SLIKA

Slika 1 - Razlika između procesa i niti	4
Slika 2 - Poređenje paralelnog i serijskog (sekvencijalnog) računanja	5
Slika 3 - Sekvencijalna verzija sabiranja elemenata velike liste.....	9
Slika 4 - Paralelna verzija sabiranja elemenata velike liste	9
Slika 5 - Sekvencijalna verzija izračunavanja kvadrata brojeva.....	10
Slika 6 - Paralelna verzija izračunavanja kvadrata brojeva	11
Slika 7 - Sekvencijalna verzija provjere prostih brojeva	12
Slika 8 - Paralelna verzija provjere prostih brojeva.....	13
Slika 9 - Sekvencijalna verzija pretrage elemenata u velikoj listi	14
Slika 10 - Paralelna verzija pretrage elemenata u velikoj listi.....	15
Slika 11 - Sekvencijalna verzija sortiranja više nezavisnih lista	16
Slika 12 - Paralelna verzija sortiranja više nezavisnih lista.....	17
Slika 13 - Sekvencijalna verzija brojanja riječi u tekstovima.....	18
Slika 14 - Paralelna verzija brojanja riječi u tekstovima	19
Slika 15 - Sekvencijalna verzija rada sa više tekstualnih fajlova	20
Slika 16 - Paralelna verzija rada sa više tekstualnih fajlova	21
Slika 17 - Sekvencijalna verzija simulacije mrežnih zahtjeva.....	22
Slika 18 - Paralelna verzija simulacije mrežnih zahtjeva	22
Slika 19 - Sekvencionalna verzija obrade slike predstavljene matricom piksela	23
Slika 20 - Paralelna verzija obrade slike predstavljene matricom piksela.....	24
Slika 21 - Sekvencijalna verzija Monte Carlo simulacije za približnu vrijednost broja pi.....	25
Slika 22 - Paralelna verzija Monte Carlo simulacije za približnu vrijednost broja pi	26
Slika 23 - Sekvencijalna verzija množenja matrica	27
Slika 24 - Paralelna verzija množenja matrica.....	28
Slika 25 - Sekvencijalna verzija izračunavanja faktoriijela	29
Slika 26 - Paralelna verzija izračunavanja faktoriijela.....	30
Slika 27 - Sekvencijalna verzija filtriranja velikog skupa podataka.....	31
Slika 28 - Paralelna verzija filtriranja velikog skupa podataka	32
Slika 29 - Sekvencijalna verzija za generisanje hash vrijednosti	33
Slika 30 - Paralelna verzija za generisanje hash vrijednosti	34
Slika 31 - Sekvencijalna verzija simulacije više nezavisnih zadataka.....	35
Slika 32 - Paralelna verzija simulacije više nezavisnih zadataka	36

CITATNI IZVORI

- [1] M. Bjelica, Programski jezik Python: skripta za studente telekomunikacija, Beograd: Elektrotehnički fakultet Univerziteta u Beogradu, 2018.
- [2] M. Stojanović, S. Cvetković i G. Stančić, »Poređenje metoda mašinskog učenja za predikciju trenda promene finansijskih vremenskih serija,« *Zbornik radova INFOTEH-JAHORINA*, svez. 16, p. 398–401, 2017.
- [3] Python Software Foundation, »Thread states and the global interpreter lock,« Python 3 documentation, [Mrežno]. Available: <https://docs.python.org/3/c-api/threads.html>.
- [4] Python Software Foundation, »PEP 703 – Making the Global Interpreter Lock Optional in CPython,« Python Enhancement Proposals, [Mrežno]. Available: <https://peps.python.org/pep-0703/>.
- [5] Python Software Foundation, »The Python Standard Library: Concurrent Execution,« 2026. [Mrežno]. Available: <https://docs.python.org/3/library/concurrency.html>.
- [6] Python Software Foundation, »threading — Thread-based parallelism,« 2026. [Mrežno]. Available: <https://docs.python.org/3/library/threading.html>.